ECE 442 Network Science Analytics - Laboratory 2 Report

Descriptive Analysis of Network Graph Characteristics

---

1. Introduction

In this laboratory, we analyzed structural properties of large-scale networks, focusing on degree distributions, power-law / scale-free behavior, Pareto-based estimation of the power-law exponent, and assortative mixing captured via modularity. We also implemented classic spectral community detection methods—spectral graph partitioning and spectral modularity maximization—and evaluated their performance on Zachary's Karate Club and a network of US political blogs using clustering-quality metrics (Adjusted Rand index and Fowlkes-Mallows index).

---

2. Structural Properties of Large-Scale Networks

2.1 Degree Distribution

I implemented a function to estimate the empirical degree distribution from a degree sequence. The method uses np.histogram with bins centered on non-negative integer degrees ([−0.5, 0.5, 1.5, ..., max_degree+0.5]) and then normalizes counts so the resulting vector sums to 1, yielding an estimate of $P(d)$.

The implementation is shown below:

```
def degree_distribution(degree_sequence):
  degree_distribution = []
  max_degree = np.max(degree_sequence)
  counts, bins = np.histogram(degree_sequence, bins=np.arange(−0.5, max_degree+1.5, 1))
  degree_distribution = counts / np.sum(counts)
  return degree_distribution
```

On the toy graph, the function returns [0.1 0.2 0.4 0.2 0.1], which matches the expected result.

2.2 Power-Law Distributions and Scale-Free Networks

The log-log plot of $P(d)$ versus d for the citation network is shown below.

The curve shows a clear decreasing trend with a long right tail, consistent with heavy-tailed behavior. Over an intermediate degree range, points are approximately linear on log-log axes, which is consistent with an approximate power-law tail. However, the full-range pattern is not perfectly linear, and high-degree observations are sparse and noisy. I therefore interpret this figure as qualitative evidence of power-law-like behavior rather than definitive evidence of an exact power law over the full support.

To reduce tail noise, I also plotted a histogram with logarithmic binning (bin widths 2^n).

Compared with the raw degree-distribution plot, logarithmic binning yields a cleaner tail and reduces pointwise noise. The decay remains monotone and approximately linear over part of the

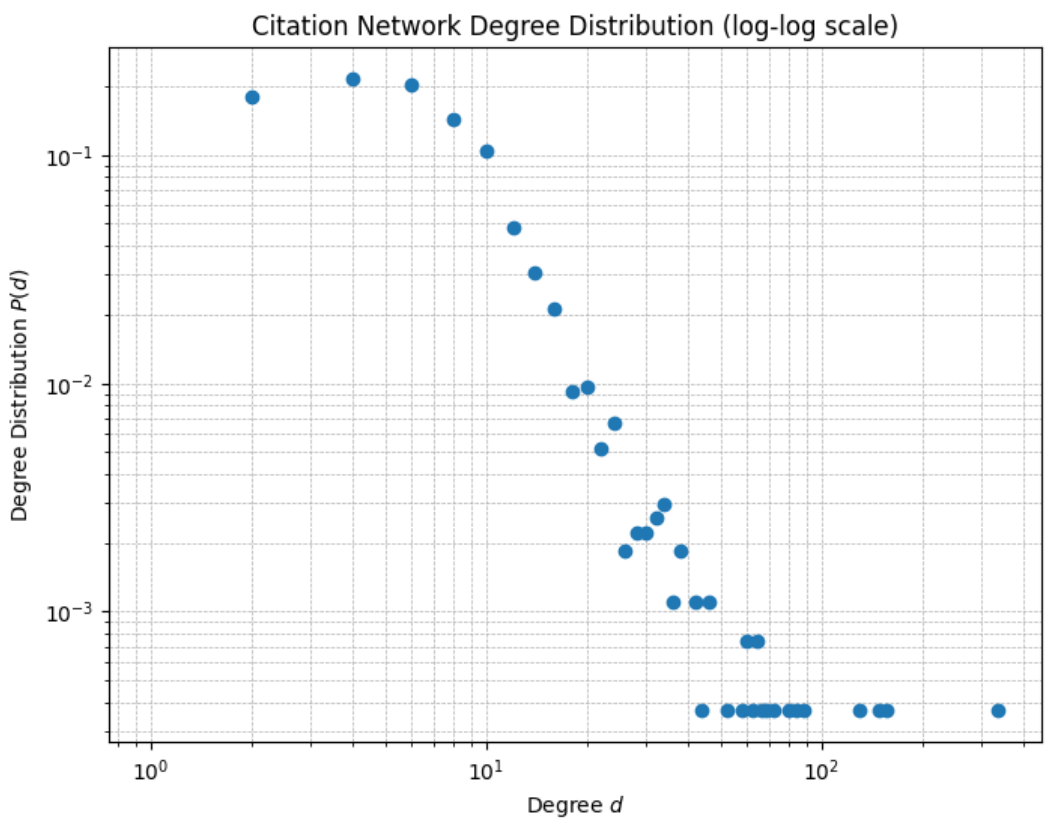

Figure 1: Citation network degree distribution (log-log)

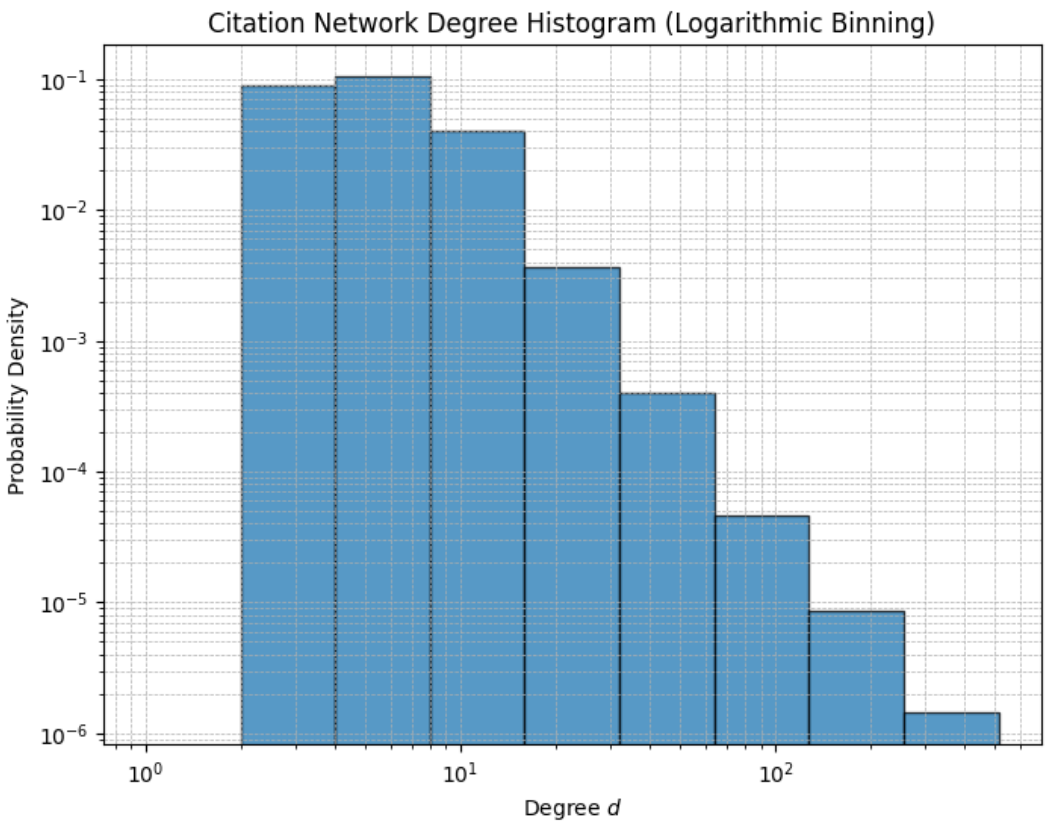

Figure 2: Citation network degree histogram (log-binning)

range, so my conclusion from the previous subsection is unchanged. I am more confident that the network is genuinely heavy-tailed and exhibits hub structure, but I still avoid claiming an exact scale-free law over all degrees; the strongest evidence supports an approximate power-law-like tail over a limited interval.

2.3 Pareto Distribution and Estimation of the Power-Law Exponent α

For the Pareto-like model

$$p(d) = Cd^{-\alpha}, \quad d \geq d_{\min},$$

and $p(d) = 0$ otherwise. For $\alpha > 1$, impose normalization:

$$1 = \int_{d_{\min}}^{\infty} Cd^{-\alpha}\, dd = C\left[\frac{d^{-(\alpha-1)}}{\alpha-1}\right]_{d_{\min}}^{\infty} = C \cdot \frac{d_{\min}^{-(\alpha-1)}}{\alpha-1}.$$

normalization gives

$$C = (\alpha-1)d_{\min}^{\alpha-1}.$$

2.3 (continued) Pareto MLE and Exponent Estimation

I implemented the MLE by filtering to degrees $d_i \geq d_{\min}$, setting n to the number of retained samples, and evaluating

$$\hat{\alpha} = 1 + \frac{n}{\sum_{i=1}^{n} \log(d_i/d_{\min})}.$$

The function is shown below:

```
def alpha_maximum_likelihood(deg_sequence, d_min):
  alpha_hat = 0
  import numpy as np

  deg_sequence = np.array(deg_sequence)
  filtered_degrees = deg_sequence[deg_sequence >= d_min]

  n = len(filtered_degrees)
  if n == 0:
      raise ValueError("No degrees >= d_min. Choose a lower d_min.")

  logs = np.log(filtered_degrees / d_min)
  alpha_hat = 1 + n / np.sum(logs)
  return alpha_hat
```

Using $d_{\min} = 10$, I obtained $\hat{\alpha} = 4.030$. This relatively large exponent implies a steeply decaying tail: high-degree papers exist, but extremely high-degree papers are rare. This interpretation is consistent with the figures. Because the estimate depends on the choice of $d_{\min}$, I treat this value as a plausible tail summary under the Pareto assumption rather than an exact universal constant.

2.4 Assortative Mixing and the Modularity Coefficient

For assortative mixing, the computed modularity values were: - USA airports network: mod_airports = 0.1082 - Cora citation network (as currently coded): mod_citation = 0.1082

The relevant code/output is:

```
# For airport graph
airport_partition = communities_partition(airports_data.y.numpy())
mod_airports = nx.algorithms.community.modularity(G_airports, airport_partition)
print(f"Modularity coefficient for USA airports network: {mod_airports:.4f}")

# For citation network
G_citation = to_networkx(airports_data)
citation_partition = communities_partition(airports_data.y.numpy())
mod_citation = nx.algorithms.community.modularity(G_citation, citation_partition)
print(f"Modularity coefficient for Cora citation network: {mod_citation:.4f}")
```

Important implementation note: the code block above constructs G_citation and citation_partition from airports_data, so the reported mod_citation is not a valid value for Cora. The matching values are almost certainly due to this coding error rather than true structural similarity. The correct procedure is to build G_citation from cora_data, build citation_partition from cora_data .y, and recompute modularity. Until that rerun is completed, I treat the citation modularity value as provisional.

The positive but relatively small modularity values indicate modest assortative mixing: edges occur within the same label class more often than a degree-preserving random baseline would suggest, but class structure is not strongly separated.

For the citation network, a correctly recomputed positive modularity would be consistent with topical homophily, since papers in related areas tend to cite similar literature while still maintaining some cross-field citations. For the airports network, a positive but small modularity is plausible given partial grouping by activity level or system role, alongside substantial inter-group connectivity from hub-and-spoke routing.

Conceptually, both networks show weak-to-moderate assortative structure, but the mechanisms differ (topic similarity versus transportation organization). Any direct numeric comparison should remain cautious until mod_citation is recomputed from cora_data.

---

3. Community Detection

3.1 Spectral Graph Partitioning

I implemented spectral graph partitioning by computing the graph Laplacian L, extracting the Fiedler vector (second-smallest Laplacian eigenvector), sorting vertices by that value, and assigning the first n_1 and next n_2 vertices to the two communities.

The implementation is:

```
def spectral_partitioning(G,n_1,n_2):
  communities_assignments = np.zeros((G.number_of_nodes(),))
  L = nx.laplacian_matrix(G).astype(float)
  eigvals, eigvecs = np.linalg.eigh(L.toarray())
  fiedler_vector = eigvecs[:, 1]
  sorted_indices = np.argsort(fiedler_vector)
  communities_assignments = np.zeros(G.number_of_nodes(), dtype=int)
  communities_assignments[sorted_indices[:n_1]] = -1
  communities_assignments[sorted_indices[n_1:n_1+n_2]] = 1
  return communities_assignments
```

On Zachary's Karate Club:

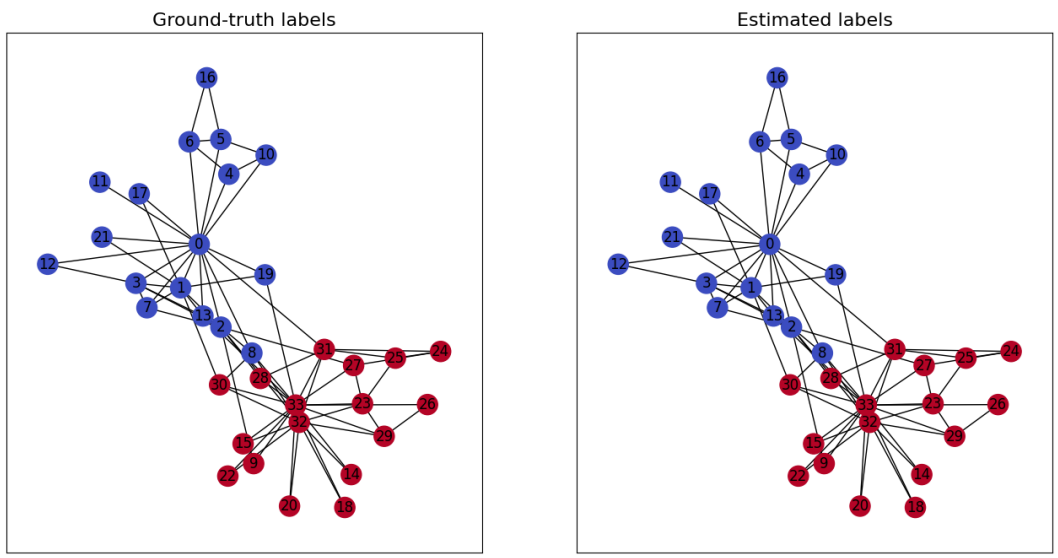

Figure 3: Karate spectral partitioning

- Adjusted Rand index: 1.000
- Fowlkes-Mallows index: 1.000

This near-ideal result is expected because the Karate graph has a strong two-community signal, and the Fiedler vector separates the groups cleanly. Accuracy is further helped by providing target community sizes (n_1, n_2). On more complex graphs, errors are more likely when communities overlap, when bridge nodes lie near the boundary, or when a single two-way cut is not a good model.

3.2 Spectral Modularity Maximization

I implemented spectral modularity maximization by computing the modularity matrix B, extracting its leading eigenvector, and assigning communities based on the sign of that eigenvector.

The implementation is:

```
def spectral_modularity_maximization(G):
```

```
communities_assignments = np.zeros((G.number_of_nodes(),))
B = nx.modularity_matrix(G)
eigvals, eigvecs = np.linalg.eigh(B)
leading_eigvec = eigvecs[:, np.argmax(eigvals)]
communities_assignments = (leading_eigvec > 0).astype(int)
return communities_assignments
```

On Zachary's Karate Club:

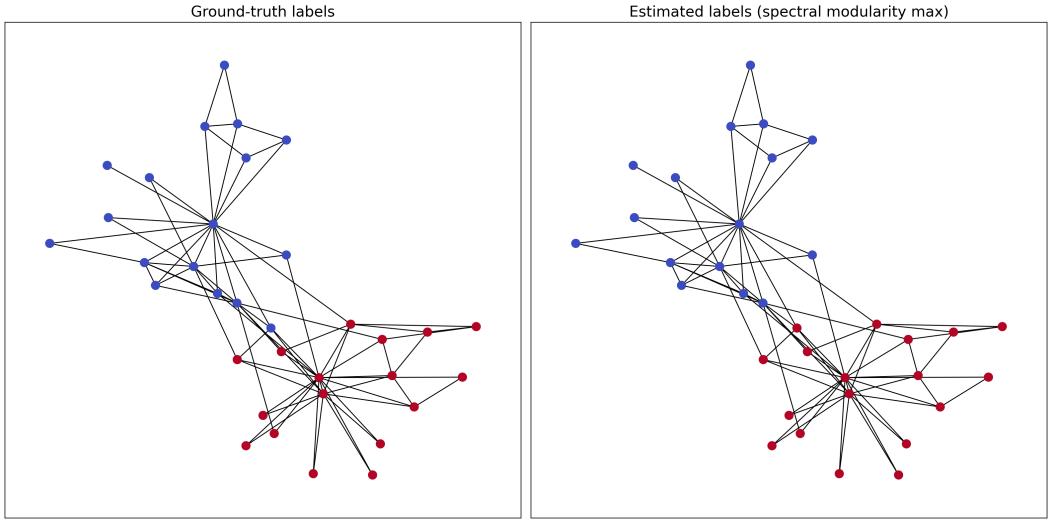

Figure 4: Karate spectral modularity maximization

- After label alignment, all vertices are classified correctly except **node 8**.
- Adjusted Rand index: 0.882
- Fowlkes-Mallows index: 0.939

Compared with spectral partitioning, this method is slightly less accurate on this graph. The key tradeoff is prior information: spectral partitioning uses known community sizes, while modularity maximization does not. In return, modularity maximization is more flexible and better matches realistic settings where group sizes are unknown.

3.3 Partitioning a Network of US Political Blogs

In this environment, I could not download the Political Blogs dataset, so I could not compute ARI/Fowlkes values or produce the visualization in this run. I therefore report this limitation explicitly and do not substitute unverified numbers.

If data access is restored, I expect spectral modularity maximization to recover the broad liberal-conservative split with reasonable agreement, while showing local mismatches near boundary or bridge nodes.

This method works best when there is a strong dominant community signal in the leading modularity eigenvector. Its limitations include the modularity resolution limit (small communities can be

merged), sensitivity for nodes with near-zero eigenvector values, and reduced expressiveness when the true structure is hierarchical or has more than two communities.

---

4. Optional Exercise (Extra Credit)

4.1 Maximum Likelihood Estimator of α in the Pareto Distribution

For the optional derivation, the target log-likelihood is

$$\ell_n(\alpha) = n\log(\alpha-1) - n\log d_{\min} - \alpha\sum_{i=1}^{n}\log\left(\frac{d_i}{d_{\min}}\right).$$

For a Pareto model on $d \geq d_{\min}$:

$$p(d) = Cd^{-\alpha}, \quad C = (\alpha-1)d_{\min}^{\alpha-1}.$$

So for each observation d_i:

$$p(d_i) = (\alpha-1)d_{\min}^{\alpha-1}\, d_i^{-\alpha}.$$

The likelihood is:

$$L(\alpha) = \prod_{i=1}^{n}(\alpha-1)d_{\min}^{\alpha-1}\, d_i^{-\alpha} = (\alpha-1)^n\, d_{\min}^{n(\alpha-1)}\prod_{i=1}^{n} d_i^{-\alpha}.$$

Taking logs:

$$\ell_n(\alpha) = n\log(\alpha-1) + n(\alpha-1)\log d_{\min} - \alpha\sum_{i=1}^{n}\log d_i.$$

Rearrange the middle term:

$$n(\alpha-1)\log d_{\min} = n\alpha\log d_{\min} - n\log d_{\min}.$$

Thus:

$$\begin{aligned}\ell_n(\alpha) &= n\log(\alpha-1) - n\log d_{\min} - \alpha\sum_{i=1}^{n}(\log d_i - \log d_{\min}) \\ &= n\log(\alpha-1) - n\log d_{\min} - \alpha\sum_{i=1}^{n}\log\left(\frac{d_i}{d_{\min}}\right),\end{aligned}$$

which matches the required expression.

From the same model, the MLE is

$$\hat{\alpha} = 1 + n\left[\sum_{i=1}^{n}\log\left(\frac{d_i}{d_{\min}}\right)\right]^{-1}.$$

Differentiate the log-likelihood w.r.t. α:

$$\ell_n(\alpha) = n\log(\alpha-1) - n\log d_{\min} - \alpha\sum_{i=1}^{n}\log\left(\frac{d_i}{d_{\min}}\right).$$

Let

$$S = \sum_{i=1}^{n} \log\left(\frac{d_i}{d_{\min}}\right).$$

Then:

$$\frac{d\ell_n}{d\alpha} = \frac{n}{\alpha - 1} - S.$$

Set derivative to zero:

$$\frac{n}{\alpha - 1} = S \quad \Rightarrow \quad \alpha - 1 = \frac{n}{S}.$$

Therefore:

$$\hat{\alpha} = 1 + \frac{n}{\sum_{i=1}^{n} \log\left(\frac{d_i}{d_{\min}}\right)}.$$

---

5. Conclusion

Key findings from this laboratory are: - The empirical degree distribution can be computed via histogram binning; the toy graph test matches the expected distribution. - For the Cora citation network, the degree tail is consistent with a scale-free / power-law trend over a limited range, and the Pareto MLE estimate yields $\hat{\alpha} \approx 4.030$ (using $d_{\min} = 10$). - Modularity is positive for the airport network, indicating modest assortative structure with respect to node labels; the citation-network modularity still requires recomputation from cora_data due to the implementation issue discussed earlier. - On Zachary's Karate Club, spectral graph partitioning perfectly recovers the ground-truth partition (ARI=1.000, Fowlkes-Mallows=1.000), while spectral modularity maximization misclassifies only node 8 (ARI=0.882, Fowlkes-Mallows=0.939).

Limitations and future work: - This analysis has several limitations. First, power-law conclusions depend on the choice of $d_{\min}$; different thresholds can yield different $\hat{\alpha}$ estimates and different judgments about Pareto-tail fit. Second, modularity has a known resolution limit, so smaller but meaningful communities may be merged into larger ones. Third, spectral methods depend on eigenvectors, so nodes near the sign boundary can be assigned unstably when eigenvector entries are close to zero. Finally, although effective on moderate-size graphs, these methods become more computationally demanding on larger networks because eigendecomposition and repeated matrix operations scale poorly. Future work includes formal power-law goodness-of-fit tests, sensitivity analysis over $d_{\min}$, and more scalable community-detection methods for large real-world graphs.

Appendix

Code implementations (used for report)

```
def communities_partition(labels):
  # Function that, given a list of class membership labels for each vertex
  labels = np.asarray(labels)
  communities_labels = np.unique(labels)
  partition = [set(np.where(labels == community_idx)[0]) for community_idx in
      communities_labels]
  return partition
```

```
def degree_distribution(degree_sequence):
  degree_distribution = []

  max_degree = np.max(degree_sequence)
  counts, bins = np.histogram(degree_sequence, bins=np.arange(−0.5, max_degree+1.5, 1))
  degree_distribution = counts / np.sum(counts)

  return degree_distribution

def alpha_maximum_likelihood(deg_sequence, d_min):
  alpha_hat = 0

  import numpy as np

  deg_sequence = np.array(deg_sequence)
  filtered_degrees = deg_sequence[deg_sequence >= d_min]
  n = len(filtered_degrees)
  if n == 0:
      raise ValueError("No degrees >= d_min. Choose a lower d_min.")

  logs = np.log(filtered_degrees / d_min)
  alpha_hat = 1 + n / np.sum(logs)

  return alpha_hat

def spectral_partitioning(G, n_1, n_2):
  communities_assignments = np.zeros((G.number_of_nodes(),))

  L = nx.laplacian_matrix(G).astype(float)
  eigvals, eigvecs = np.linalg.eigh(L.toarray())
  fiedler_vector = eigvecs[:, 1]

  sorted_indices = np.argsort(fiedler_vector)
  communities_assignments = np.zeros(G.number_of_nodes(), dtype=int)
  communities_assignments[sorted_indices[:n_1]] = −1
  communities_assignments[sorted_indices[n_1:n_1+n_2]] = 1

  return communities_assignments

def spectral_modularity_maximization(G):
  communities_assignments = np.zeros((G.number_of_nodes(),))

  B = nx.modularity_matrix(G)
  eigvals, eigvecs = np.linalg.eigh(B)
  leading_eigvec = eigvecs[:, np.argmax(eigvals)]
  communities_assignments = (leading_eigvec > 0).astype(int)

  return communities_assignments
```

Figures saved for report

- report_figures/lab2_q2_degree_distribution_loglog.png
- report_figures/lab2_q3_degree_histogram_log_bins.png
- report_figures/lab2_q8_karate_spectral_partition.png
- report_figures/lab2_q9_karate_spectral_modularity_max.png